
Reproducing StraightPCF in Jittor

Point-Cloud Denoising and the Cost of Following the Paper

Yi Hou

University of Chinese Academy of Sciences
Beijing, China
houyi25@mails.ucas.ac.cn

Abstract

This report documents an attempt to reproduce StraightPCF (CVPR 2024), a point-cloud filtering method, in Jittor for Track 2 of the 6th Jittor AI Challenge. Our best result, 67.44/100 on a held-out split under the competition’s formula, and a national top-100 placement, came from a pipeline that deviates from the paper as much as it follows it: two velocity modules trained on the competition’s own noisy clouds, with neither the coupling term nor the distance module. Rewriting it toward the paper’s prescription, after the submission was frozen, cost 2.65 points; the change we can pin that on is the noise model, where a module trained only on Gaussian noise scores 1.01/100 against the competition’s Laplace noise. The evaluation split was nine meshes, so the number itself carries a wide error bar. The other half of the report is the Jittor implementation this family of methods needs: chunked KNN, GPU farthest-point sampling, scatter-mean edge convolutions, MPI data parallelism, and an evaluation harness.

1 Introduction

The 6th Jittor AI Challenge is a competition organized around Jittor, a JIT-compiled deep-learning framework developed at Tsinghua University. Track 2 asked entrants to denoise point clouds sampled from ShapeNet meshes: given a noisy cloud, predict a displacement for every point so that the displaced cloud lies close to the original surface. Entries had to run in Jittor on a provided ShapeNet subset. This report describes a solo entry: a reproduction of StraightPCF [1], a CVPR 2024 method that models filtering as an optimal-transport-like flow with a learned constant velocity field.

The system removes most of the noise and was accepted for scoring, but it does not reach the quality the paper reports, and it placed in the top 100 nationally. The submitted model was also not the paper’s method: it trained on the competition’s own noisy clouds, without the coupling term or the distance module. The rewrite that added them, after the submission was frozen, scored 2.65 points lower.

Sections 2–3 define the task and the method as implemented, Section 4 covers the Jittor engineering, and Section 5 reports the measured results. Section 6 then diagnoses the shortfall, Section 7 covers the qualification round, and Sections 8–9 close with lessons and reproducibility.

2 Task, Data, and Metric

Task. The input is a noisy point cloud $\{p_i + n_i\}_{i=1}^N$ where the clean points lie on a shape surface normalized into the unit ball and the noise is Laplace-distributed with $\sigma \in [0.005, 0.020]$. The output is a point cloud with *exactly* the same number of points, float32, shape $(N, 3)$; points are expected to move back toward the surface, and the organizers check the point count.

Data. Training uses approximately 20,000 ShapeNet meshes (.obj); the test set is a collection of pre-noised clouds (.npy). Because the test set ships without clean references, we built a self-evaluation harness (self_eval.py) that reserves a fraction of the training meshes (default 10%), samples points from each mesh, adds Laplace noise at the midpoint of the competition range, runs the full inference pipeline, and computes the competition metrics against the mesh. All numbers in Section 5 come from this harness.

Metric. Both metrics are means of *squared* nearest-neighbour distances. Chamfer distance is symmetric,

$$\text{CD}(A, B) = \frac{1}{|A|} \sum_{a \in A} \min_{b \in B} \|a - b\|_2^2 + \frac{1}{|B|} \sum_{b \in B} \min_{a \in A} \|b - a\|_2^2, \quad (1)$$

computed against the clean point set; P2S is one-directional and measured to the mesh surface (via a BVH, with a vertex-only fallback),

$$\text{P2S}(A, M) = \frac{1}{|A|} \sum_{a \in A} \min_{s \in M} \|a - s\|_2^2. \quad (2)$$

Both clouds are normalized to the unit ball before measuring. The competition score maps each metric to a per-sample score and averages:

$$s_i = \text{clamp} \left(100 \left(1 - \frac{m_i^{\text{pred}}}{m_i^{\text{noisy}}} \right), 0, 100 \right), \quad \text{Score} = \frac{1}{2} \overline{s^{\text{CD}}} + \frac{1}{2} \overline{s^{\text{P2S}}}. \quad (3)$$

The score averages per-sample ratios, not the ratio of pooled means: a method can reduce pooled P2S by 67% and still show a P2S sub-score of 83, because samples where the prediction is far better than the noise pull the mean upward while a few hard samples are clamped at 0. We report both views in Section 5.

3 Method as Implemented

The repository holds two pipelines that are not the same model trained the same way. The first produced the 67.44 submission in May 2026. The second is the code as it stands after a July 2026 rewrite that moved toward the paper’s construction. They differ in the noise model, the regression target, the encoder width, and in whether the coupling term and the distance module are used at all. The rewrite scored 2.65 points lower, and Section 6 argues that the noise model is why. Each subsection describes the submitted pipeline first and marks what the shipped code changed.

3.1 StraightPCF

StraightPCF [1] treats denoising as a transport problem: points are moved from the noisy distribution to the clean one along approximately straight paths, which makes few-step integration sufficient at inference. A velocity module v_θ is trained to predict a displacement that is *constant along the path*: for a pair (X_0, X_1) of high-noise and clean samples, the regression target is $X_1 - X_0$. Two velocity modules are coupled: VM1 moves the cloud one step, VM2 refines from the moved state, and a distance module d_ϕ predicts a scalar in $(0, 1)$ that scales the step size so that points stop *on* the surface rather than overshooting it. Inference is Euler integration with $T = \text{steps} \times K$ sub-steps, $K = 2$.

3.2 Data pipeline

Each training sample starts the same way in both pipelines. From a mesh we sample 32,768 surface points and normalize the cloud to the unit ball, recording the center and scale for later denormalization. With probability 0.5 the cloud is rotated by a random Euler angle and rescaled by a factor in $[0.8, 1.2]$. We cut 8 patches of 1,000 points each, drawing a seed from the noisy cloud and taking its nearest neighbours through a KD-tree built on that same noisy cloud.

Where the pipelines part is the pair of clouds the regression is built from. The *submitted* pipeline uses the competition’s own material: X_0 is the patch of noisy points, X_1 the clean points at the same indices, and a per-point interpolation parameter $t \in (0, 1]$ gives the encoder input

$$X_t = (1 - t) X_0 + t X_1. \quad (4)$$

Table 1: Architecture of the two pipelines. Parameter counts are computed from the layer dimensions.

Component	Layers	Submitted	Shipped
Feature extraction	EdgeConv 3→32, 32→64, 96→256	152k	—
	EdgeConv 3→64, 64→128, 192→256	—	260k
Decoder	Linear 256→256→64→3 + BN + dropout(0.1)	83k	83k
Distance module	Linear 256→64→64→1, max over patch, sigmoid	—	21k
Velocity module	encoder + decoder	235k	343k
Total	VM1 + VM2 (+ distance module)	470k	707k

The regression targets are then real displacements — $X_1 - X_0$ for VM1, and the residual $X_1 - X_{t_1}$ for VM2 — which is the motion the competition actually asks for. All three clouds are centered on the seed’s current position, keeping displacements in a patch-local frame, and each patch supervises 128 points.

The *shipped* code synthesizes the noisy sample instead: patch neighbours come from the clean cloud, and $X_0 = X_1 + \varepsilon_H$ with $\varepsilon_H \sim \mathcal{N}(0, \sigma_H^2)$, $\sigma_H = 0.02$. The interpolation and the targets keep the same form, and supervision rises to 256 points per patch. This is the paper’s construction applied to the wrong noise distribution, and Section 6 traces the consequences.

3.3 Architecture

Both pipelines use the same shape of network: a three-layer dynamic EdgeConv stack followed by an MLP decoder. Each EdgeConv layer builds a k -NN graph ($k = 16$) on its own input features, applies an MLP to the concatenation $[x_i, x_j - x_i]$ of a point and its edge vector, aggregates messages by scatter-mean, and adds a linear skip on x_i . The decoder is a small MLP with batch normalization and dropout. Table 1 gives the layer sizes for both.

The July rewrite doubled the encoder’s channel widths and added a distance module, which shares the encoder’s features, regresses a per-point scalar, reduces it by max over the patch, and applies a sigmoid following the paper’s Eq. 13 ordering. The submitted model never used the distance module. It was instantiated in the code, but a commit four days before the submission removed it from the Euler step, and the training objective never contained a distance term at all.

3.4 Training objectives

The submitted model was trained with the velocity loss alone, applied to each module:

$$\mathcal{L}_{\text{vel}} = \frac{1}{N} \sum_i \frac{1}{\sigma_{\text{dsm}}} \|v_{\theta}(X_t)_i - (X_1 - X_0)_i\|_2^2, \quad (5)$$

with $\sigma_{\text{dsm}} = 0.01$. The $1/\sigma_{\text{dsm}}$ factor is ours, not the paper’s, and it is not harmless: it weights the velocity term by 100 relative to every other term in the July objective. VM1 is trained on the interpolation state X_t with the target $X_1 - X_0$; VM2 is trained on the state $X_{t_1} = X_0 + \frac{1}{K} v_{\theta_1}(X_0)$ that VM1 produces, with the residual target $X_1 - X_{t_1}$. The two losses sum, and there is no coupling term and no distance term.

The July pipeline adds both, taken from the paper. The coupling term is Eq. 10 with $\lambda_1 = 10$,

$$\mathcal{L}_{\text{cpl}} = \lambda_1 \|X_{t_1}^{\text{pred}} - X_{t_1}^{\text{ideal}}\|_2^2, \quad X_{t_1}^{\text{pred}} = X_t + \frac{1}{K} v_{\theta_1}(X_t), \quad t_1 = \frac{t(K-1)+1}{K}, \quad (6)$$

where $X_{t_1}^{\text{ideal}}$ is the interpolation point at t_1 , and the distance module is trained with a direct term and a chain term,

$$\mathcal{L}_d = \|d_{\phi}(X_t) - (1-t)\|_2^2 + \lambda_2 \|\bar{X}_1 - X_1\|_2^2, \quad \lambda_2 = 200, \quad (7)$$

where $\bar{X}_1$ is the result of a two-step Euler chain through VM1 and VM2 whose *only* differentiable input is d_{ϕ} ; that chain is what teaches d_{ϕ} to scale steps for convergence rather than merely regress $1 - t$.

Algorithm 1 Patch-based filtering of one noisy cloud

```
1: Input: noisy cloud  $P \in \mathbb{R}^{N \times 3}$ , patch size  $M=1000$ , seeds  $K \approx 6N/M$ 
2: Estimate noise level from mean 8-NN distance; set repeat count  $R \in \{2, 3, 4\}$ 
3: for  $r = 1$  to  $R$  do
4:    $\{s_k\} \leftarrow \text{FPS}(P, K)$  ▷ farthest-point seeds
5:    $\{P_k\} \leftarrow M\text{-NN patch of each } s_k$ ; center each patch on its seed
6:    $P'_k \leftarrow \text{Euler}_{\theta, \phi}(P_k)$  for all  $k$  (batched,  $\leq 8$  patches at a time)
7:   Re-center patches; each point takes the patch where  $\tilde{d}_k(p)$  is smallest
8:   Points with no assignment keep their previous positions
9: end for
10: return  $P$  (denormalized)
```

Training schedule. The paper trains in stages: pretrain VM1 alone, pretrain VM2 alone, couple them with $\mathcal{L}_{\text{vel}} + \mathcal{L}_{\text{cpl}}$ while the distance module is frozen, then train the distance module with the backbone frozen. The submitted model was trained in a single phase instead: both modules update in the same step, with VM2’s input computed from VM1’s current output as a constant. The July pipeline used a two-phase approximation of the paper’s schedule, and the paper’s four stages exist in the repository as `configs/task/train_vm_stage{1..4}.yaml`, run after the submission was frozen: Stages 1 and 2 completed (Figure 2a), Stage 3 was stopped after one epoch, and Stage 4 was never run.

3.5 Inference

Inference is patch-based (Algorithm 1) and identical in both pipelines apart from the step size. Encoding a full 32k-point cloud at once disagrees with the patch statistics seen in training and exceeds memory at useful batch sizes, so the network runs on overlapping patches and the results are merged. Seeds come from farthest-point sampling; each patch is the seed’s 1,000 nearest neighbours, centered on the seed. The Euler integrator runs three steps of the two modules, giving $T = 3 \times 2 = 6$ sub-steps. The submitted model takes each sub-step at a fixed size $1/T$; the shipped code scales it by d_ϕ , the distance module’s output.

Merging is a hard assignment, not a blend. For each point and each patch containing it we compute $\tilde{d}$, the point’s KNN-rank distance inside that patch normalized by the patch radius, and the point takes the prediction of the patch where $\tilde{d}$ is smallest — the patch in which it is most central. Points that fall in no patch keep their noisy positions. The whole pass is then repeated 2–4 times, with the count set from an estimate of the noise level, a deviation from the paper’s single pass; Section 6 takes it up.

4 Jitter Engineering

Jitter executes Python eagerly but builds and fuses the underlying computation graph with its own JIT, and it has no PyTorch-ecosystem counterpart for point-cloud primitives. Almost every geometric primitive this task needs — k -NN, farthest-point sampling, edge convolutions — had to be written by hand against Jitter ops. This section documents those primitives and the memory discipline they required.

4.1 Primitives

Chunked k -NN. A direct broadcast $(N, 1) \rightarrow (1, M)$ over a 1,000-point patch is fine, but k -NN is also needed against 32,768-point clouds, where the full distance matrix ($N \times M$ floats) is not. The implementation streams the reference cloud in chunks of 1,024, maintains a running top- k by concatenating each chunk’s distances with the running best and re-selecting, and records the original indices through `gather`. Peak memory scales with the chunk size rather than with the reference cloud.

Farthest-point sampling. FPS is sequential by nature: our version performs one `argmax` per selected point, and each costs a GPU→CPU synchronization. Patch seeding selects only about six points per

1,000, so the syncs are affordable there; whole-cloud downsampling selects thousands and uses a NumPy implementation instead.

Edge convolution. Jittor has no scatter-mean kernel on GPU for this layout, so EdgeConv is implemented with dense `scatter_` (`reduce='add'`) over the destination index, followed by division by the accumulated count, plus the linear skip branch. It operates directly on the $(points, k)$ edge list produced by the graph builder.

NumPy/SciPy at the edges. Patch construction, patch blending, and the evaluation metrics use `scipy.spatial.cKDTree` on the CPU — not for speed but for predictability: KNN targets computed in the graph re-entered the computational graph and inflated memory, so a later variant (the score-based detour, Section 6) computes its targets entirely on the CPU.

4.2 Memory and the computation graph

Jittor accumulates the computation graph until it is executed and collected, and our first implementation fought both out-of-memory failures and intermittent segfaults. The crash signatures pointed the wrong way: one run died with a segfault after 2,200 iterations, and a later failure blamed the batch size for running out of memory, when the device was 45.9 GB of 47 GB full with 11,000 live operators and 12,000 live variables resident. The memory was held by the graph, because a single fusion kernel could hold several hundred k-NN and edge-convolution operators at once. The fix that made long runs possible was an explicit hygiene discipline: `jt.sync_all()` (execute and release) followed by `jt.gc()` after every encoder block, after every training step, and at epoch boundaries, a cap of 8 patches per inference batch, and detaching patch outputs from the graph before blending. With the graph bounded this way, the model trains at batch size 24 with 8 patches per sample. The syncs cost throughput, but the discipline turned a training run that reliably crashed into one that completed 100 epochs unattended.

4.3 Vectorizing the reconstruction

The rule that mattered most: find the Python loop that synchronizes on every iteration and replace it with a masked tensor operation. The patch-blending reconstruction started as per-patch Python loops, and every comparison against the assignment mask crossed the GPU→CPU boundary through `.item()`, costing 3×10^5 synchronizations per cloud. Vectorizing the assignment with `jt.equal + jt.nonzero` masked scatter brought that to 3×10^2 . The same pattern recurred in FPS and in KNN.

4.4 Distributed training

Data-parallel training uses Jittor’s built-in MPI integration (`mpirun -np 6 python run.py ...`): parameters are broadcast at start via `mpi_param_broadcast`, validation losses are all-reduced, and checkpointing/logging are rank-0 only. Two practical notes: dataloaders are constructed before the CUDA context is initialized to avoid the fork-after-CUDA hazard, and the launch configuration carries `NCCL_P2P_DISABLE=1` as a workaround for peer-to-peer collective hangs on the shared node. The submission run trained at 330–377 s per epoch, and the July reruns of the same configuration took 936 s. The distance-module phase needed 1,400–1,900 s because its Euler chain adds four no-gradient encoder passes per step against two for the velocity modules alone. Stage 1, which supervises all 1,000 points per patch, ran at 813 s.

4.5 Evaluation and profiling tooling

Three scripts shaped the work more than any model change. `self_eval.py` builds a held-out split and computes the full competition metric, including the mesh-based P2S, so that checkpoint selection could target the metric rather than the loss. `profile.py` attributes wall-clock time between data loading and GPU compute and auto-probes the batch size that fits. `vis_denoising.py` renders per-sample diagnostics — displacement direction error, magnitude, and error histograms — which is how we first saw that the network’s displacements were directionally decent but mis-scaled, the symptom that later pointed at the distance module.

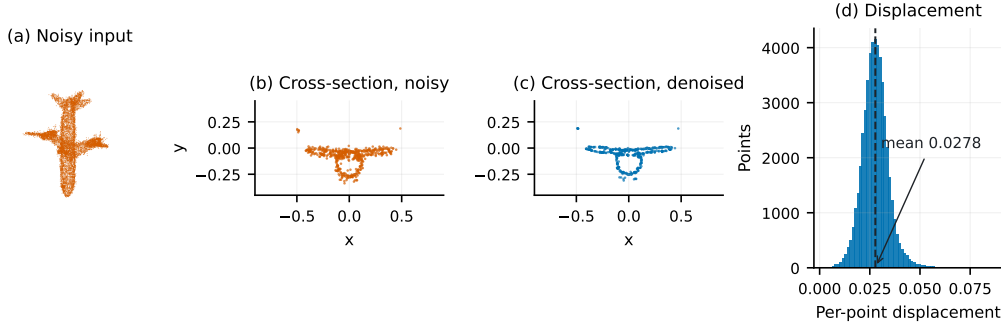


Figure 1: Filtering one test-set cloud of 50,000 points. (a) The noisy input. (b) A thin cross-section, $|z| < 0.006$: wing and fuselage read as thick, fuzzy bands. (c) The same cross-section after filtering, where the bands collapse to a surface profile. (d) The per-point displacements the network applied, mean 0.0278. No reference surface exists for this sample, so the panels show the filter’s effect rather than its accuracy; the accuracy numbers are those of Table 2.

Table 2: Headline results. Sub-scores and the final score follow the competition’s per-sample formula (Eq. 3); all figures are from the held-out self-evaluation split. The last column is the July rewrite, which changed the encoder width and the noise model alongside the training terms.

Quantity	Noisy input	Best result	July rerun
CD, pooled mean ($\times 10^{-4}$)	2.46	1.21	1.20
P2S, pooled mean ($\times 10^{-4}$)	1.96	0.64	0.72
CD sub-score	—	51.94	51.65
P2S sub-score	—	82.94	77.93
Final score	—	67.44	64.79

5 Results

5.1 Denoising quality

Table 2 gives the headline numbers for our best result: 67.44/100 on the held-out split, with the Chamfer half (51.9) weaker than the P2S half (82.9). Pooled over samples, CD falls from 2.46×10^{-4} to 1.21×10^{-4} and P2S from 1.96×10^{-4} to 6.4×10^{-5} . The gap between the pooled P2S reduction of 67% and the P2S sub-score of 82.9 is the per-sample averaging of Eq. 3 at work, as Section 2 describes.

The paper’s published accuracy is measured on Gaussian-noise benchmarks with dense point clouds; the competition’s test distribution is Laplace noise on ShapeNet, so we do not compare the two numerically.

One caveat belongs on the number itself. The harness takes its held-out set as the first 10% of the validation list, which is sorted by synset, so the split is nine meshes and not a random sample of them. The score is a mean over those nine; the second decimal is noise, and a different split would move it. Every comparison in this report is between runs scored on this same split.

5.2 Training dynamics

The staged pretraining runs are reproducible: VM1 and VM2, trained independently from different seeds with the same recipe, follow nearly identical trajectories, from 3.78 down to 0.101 over 15 epochs. Both the pretraining and the coupled run also reach their loss floors quickly: within 5 and about 40 epochs respectively. The remaining 60 epochs of the coupled run changed neither the loss nor the metric, and Section 6 argues that what kept the two from falling further was the training target rather than the optimizer.

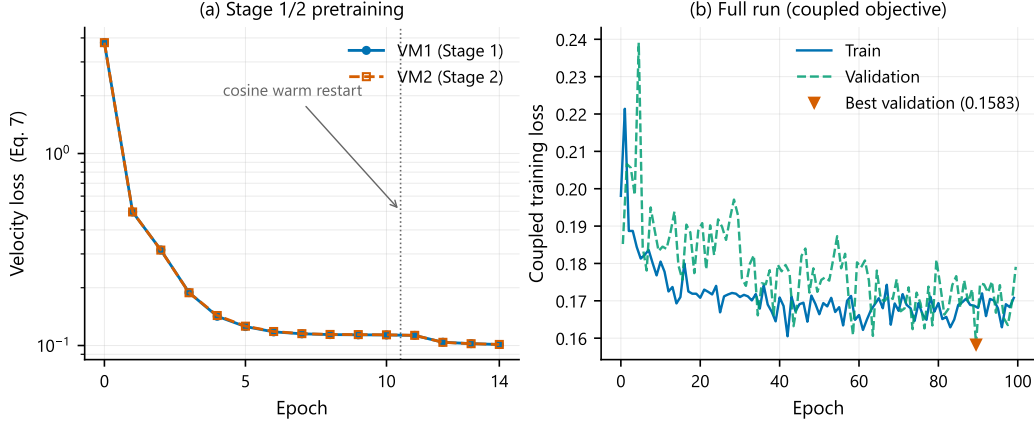


Figure 2: (a) Stage 1/2 pretraining (Eq. 7 velocity loss, one value per epoch) from the staged rerun of July 2026. The two independent runs are nearly indistinguishable, and the loss falls by a factor of 33 by the time of the cosine warm restart at epoch 11; both settle at ≈ 0.101 . (b) The full coupled training run of May 2026 that produced the 67.44 result (train and validation loss, one epoch per point). Both curves plateau by epoch ~ 40 ; the remaining 60 epochs did not improve the metric.

5.3 What changed the numbers

The competition metric improved only through changes made in sequence, and none of them was ablated in isolation; the 67.44 of Table 2 came after all of them. Three were followed by better scores. Training on 8 patches of 1,000 points per sample instead of one multiplied the supervised points per sample by eight. Moving inference from whole-cloud Euler integration to patch-based filtering removed both a train/inference statistics mismatch and the memory ceiling. Feeding the encoder the same interpolation state used in training when computing the coupled state (X_{t_1}) removed a small distribution shift. Two further changes were throughput and stability work rather than accuracy work: removing the distance module from the phase-1 forward pass, which cut encoder calls per step from three to two, and the reconstruction vectorization and sync discipline of Section 4.

6 Post-Mortem: What Failed and Why

Rewriting toward the paper cost 2.65 points. The submitted pipeline trained on the competition’s own noisy clouds: the encoder saw Laplace noise at the test amplitude, and the target was the displacement the metric asks for. The July rewrite replaced that with the paper’s synthetic construction — a clean patch plus Gaussian noise at $\sigma_H = 0.02$, the paper’s own benchmark scale — and changed four other things with it: the velocity target, the encoder width, the coupling term, and the distance module. The score fell from 67.44 to 64.79, and the loss was confined to one half of the metric: P2S dropped 5.0 points while CD moved by 0.3.

The changes are confounded, but one of them has direct evidence behind it. In the staged rerun, a single module trained on pure Gaussian noise, its velocity loss down to 0.101, scores 1.01/100 when evaluated on the competition’s Laplace noise: CD sub-score 0.00, P2S sub-score 2.03. Scoring is relative to the noisy input, so 1.01 is the floor — the module left the cloud essentially unchanged. A network that has only ever seen one noise family has nothing to offer the other, and the rewrite invited exactly that failure.

The submitted pipeline avoided it by accident. It was never a faithful reproduction — single-phase training, no coupling term, no distance module, and a training pair drawn from the real test distribution instead of synthesized — and for this competition that accident was the better choice.

The hypothesis we had to drop. For two months we attributed the lost points to the distance module. The story fit the paper: the module is meant to be trained last with the backbone frozen, our schedule trained it briefly and then unfroze the backbone, and a step-size regulator trained against shifting features would break the surface term (P2S) while leaving the direction of motion (CD) intact

— which is the signature we measured. It was coherent and it was wrong. The submitted model never used a distance module, so no property of that module can explain a regression that appeared only in its successor. We had reasoned about the code in front of us rather than the code that produced the number.

Inference repetition. Our inference repeats the patch filter 2–4 times depending on estimated noise. Repeating the filter raises the score, because iterated Euclidean projections onto an imperfectly straight field behave like a curved path, but the reported quality then comes partly from inference compute rather than from the velocity field. Under the paper’s single pass we could not reproduce the paper’s numbers.

A plateau we did not diagnose in time. Loss floors at epoch 5 (pretraining) and ~ 40 (coupled) with no metric movement afterward should have redirected us from learning-rate tuning to the objective itself: the loss is a proxy, and nothing guaranteed that driving it to its floor would optimize CD/P2S. The metric harness that could have told us this earlier was itself built mid-competition — a sequencing mistake.

The score-based detour. In the final weeks we implemented a ScoreDenoise-style [2] variant reusing the same encoder/decoder: the target becomes the empirical score $\text{NN}(X_t, X_{\text{clean}}) - X_t$ and inference becomes annealed Langevin dynamics. It is distribution-agnostic — it assumes neither a noise family nor straight paths — which is the property our Laplace/Gaussian mismatch needed. The implementation hit memory problems on the $O(N^2)$ nearest-neighbour target over 1,000-point patches, solved first by chunking and finally by computing targets on the CPU with `cKDTree`; the code exists in the repository (`src/model/score_vm.py`) and was never trained to a comparable state before the deadline. It consumed the last stretch of the competition and produced no scored result.

Process. Three process failures, in decreasing order of cost: the July rewrite replaced a working training distribution without keeping the old one as a control, so the points went with it and the cause had to be reconstructed afterwards; the evaluation split was nine meshes, too small for the scores or the gaps between variants to carry two decimals; and the harness for the official metric arrived mid-competition, after weeks spent optimizing a proxy loss.

7 The Qualification Round: PCT for ModelNet40

The track opened with a qualification round: classify ModelNet40 shapes from 2,048 points into 40 categories, in Jittor, with a test-accuracy threshold of 80% to advance. Our entry ranked 12th in the round.

7.1 Model

The final model follows the full architecture of the PCT paper [3] ($\sim 2.9\text{M}$ parameters): a convolutional stem ($3 \rightarrow 64 \rightarrow 64$), two sample-and-group stages (farthest-point sampling $1024 \rightarrow 512 \rightarrow 256$, k -NN with $k = 32$, local feature aggregation), four offset-attention layers with position encoding and residual connections, a concatenated skip into a $1280 \rightarrow 1024$ projection, and a max-pooled classifier ($1024 \rightarrow 512 \rightarrow 256 \rightarrow 40$).

Two details are worth noting. The four attention layers each carry their own positional projection rather than sharing one; we have no ablation that isolates the effect of this change. And the low-level operators are not ours: `rf_ops.py` ports and adapts the index, ball-query and k -NN CUDA kernels of PointNet++ [4] to Jittor, alongside a farthest-point sampler.

7.2 Training recipe, and what actually helped

Training used AdamW at $\text{lr } 0.01$ with weight decay 10^{-4} and a warmup-cosine schedule over 300 epochs, batch size 32, label smoothing 0.1, dropout 0.5, and weight EMA. The augmentation set scaled the cloud randomly to $[0.67, 1.5]$, translated it by ± 0.2 , dropped 10–40% of the points, and added Gaussian jitter at $\sigma = 0.01$; rotation augmentation appeared in earlier versions and was removed during tuning. Validation used a fixed 90/10 split.

The version history is short. A minimal PCT already cleared the threshold at about 80% accuracy: four attention layers, no sample-and-group stages, rotation-only augmentation, SGD with cosine annealing. An experiment that added a geometric attention bias, rescaling pairwise Euclidean distances by a learned factor, underperformed and was dropped. What moved the result was the hierarchical sample-and-group stages and the widened augmentation set. Per-version accuracies were not recorded.

The data-side work produced the gains here — augmentation ranges, the sample-and-group stages, the validation split — and architecture variants did not. On the main track the emphasis inverted: weeks went into architecture and schedule variants while the noise model and the evaluation harness were fixed late.

7.3 Dropout that ignores the training flag

Jittor’s dropout layers did not follow the module’s training flag reliably in our version. The final training script works around this by walking the module tree and setting every dropout layer explicitly (`_set_dropout_train`) at the start of each epoch and before validation. Until that fix, runs trained with dropout in an unintended state, and nothing in the loss curves or the logs would have announced it.

8 Lessons Learned

- *The training distribution is part of the method.* Our best model trained on the competition’s own noisy clouds; rewriting it toward the paper’s synthetic Gaussian data cost 2.65 points. When the method’s math assumes a noise family, the family is not a detail.
- *A causal story built from the code in front of you is not evidence.* We spent two months blaming a poorly-trained distance module for a regression that the submitted model — which never had one — could not have caused. The story fit; the code did not.
- *A loss floor is not a metric result.* Both losses reached clean floors and the metric did not follow. The held-out harness for the real metric belonged in the first week.
- *Small evaluation splits hide the truth.* Nine meshes cannot resolve a 2.65-point change, nor justify two decimals.
- *Inference tricks that compensate hide what is broken.* Repeating the filter raised the score and concealed a velocity field that was not straight enough.
- *Memory discipline is part of the model in Jittor.* `sync_all/gc` placement, chunked KNN, and patch-batch caps made long runs possible and change what can be trained, not just how fast.
- *Look for per-iteration `.item()` calls first.* They dominated reconstruction until vectorized; the pattern repeated in FPS and KNN.
- *MPI data parallelism was cheap; the primitives were not.* Jittor’s broadcast/all-reduce integration worked in an afternoon, and the time went to KNN, memory, and the metric.

9 Reproducibility

The repository contains the training and evaluation pipeline. Training and inference run through `denoise/run.py` with per-task YAML configs under `denoise/configs/`; those configs describe the July pipeline, and the submitted pipeline’s versions of them survive only in the git history, in the May 2026 commits that precede the July reorganization. Evaluation is `self_eval.py` (held-out split, full metric) and `evaluate.py` (official-style scoring); diagnostics are `profile.py` and `vis_denoising.py`. Figures and tables in this report are generated by `report/make_figures.py` from the run logs (the May run log is included under `report/data/`). The dataset — ShapeNet meshes and the pre-noised test clouds — is provided by the competition organizers and is not redistributed here, and neither are the split lists under `datalist/`, which the scripts expect to be in place.

Acknowledgement of scope. The task definition, dataset, and evaluation protocol belong to the organizers of the 6th Jittor AI Challenge (Track 2). The StraightPCF method and its equations are due to de Silva Edirimuni et al. [1]; the encoder and decoder follow the authors' reference implementation, and the classification round's low-level kernels are ported from PointNet++ [4]. The Jittor implementation, the experiments, the evaluation harness, and all measurements and errors in this report are my own.

References

- [1] Dasith de Silva Edirimuni, Xuequan Lu, Gang Li, Lei Wei, Antonio Robles-Kelly, and Hongdong Li. StraightPCF: Straight point cloud filtering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [2] Shitong Luo and Wei Hu. Score-based point cloud denoising. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [3] Meng-Hao Guo, Jun-Xiong Cai, Zheng-Ning Liu, Tai-Jiang Mu, Ralph R. Martin, and Shi-Min Hu. PCT: Point cloud transformer. *Computational Visual Media*, 7(2):187–199, 2021.
- [4] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *CVPR*, 2017.